

Spring Boot Configuration & Profiles

Concise revision notes
Bullet points • Java examples • Flow diagrams

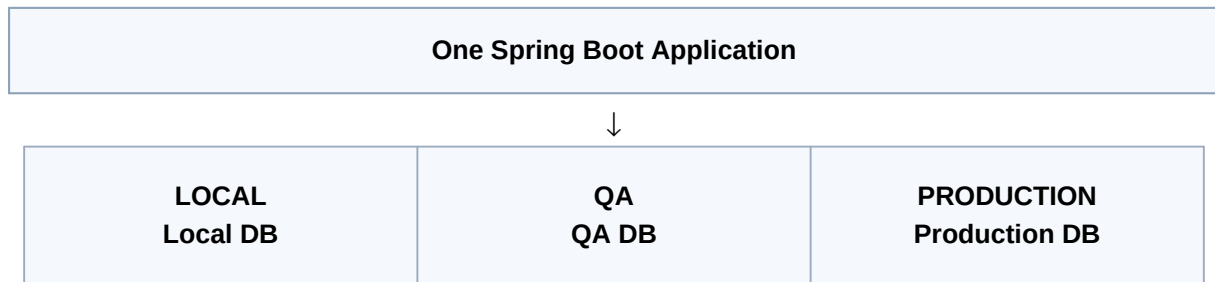
Configuration values → Active profile → Matching beans → Application runs

Quick-reference study notes

1. What is Configuration?

- **Code:** business logic of the application.
- **Configuration:** values that can change between environments.
- Examples: database URL, server port, API keys, and active profile.

2. Why Configuration?



Goal: change configuration without changing application code.

3. application.properties and @Value

Example configuration:

```
server.port=8081
app.name=MyApplication
app.features=login,signup,payment
```

Read a value in Java:

```
@Value("${app.name}")
private String appName;
```

- `${app.name}` resolves to the configured value.
- Avoid hardcoding environment-specific values in Java code.

4. application.properties vs application.yml

Properties format:

```
server.port=8081  
spring.application.name=demo
```

YAML format:

```
server:  
  port: 8081  
spring:  
  application:  
    name: demo
```

- Both configure a Spring Boot application.
- YAML represents nested settings using indentation.

5. Internal vs External Configuration

INTERNAL	EXTERNAL
Stored inside the application project.	Supplied outside the packaged app, e.g. environment variables.

- Internal configuration is stored with the project.
- External configuration is supplied outside the packaged application.

6. Profiles

Profiles let one application use different settings for different environments.

```
application.properties  
application-dev.properties  
application-qa.properties  
application-prod.properties
```

Example: application-dev.properties

```
server.port=8081
```

Example: application-prod.properties

```
server.port=8080
```

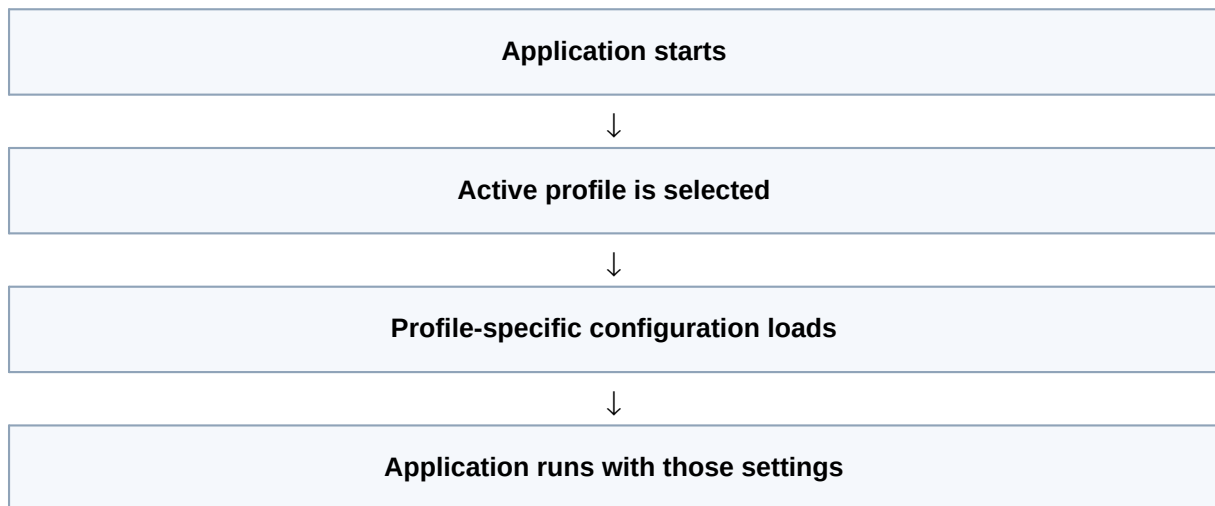
Activate a profile:

```
spring.profiles.active=dev
```

Or select it at launch:

```
java -jar app.jar --spring.profiles.active=prod
```

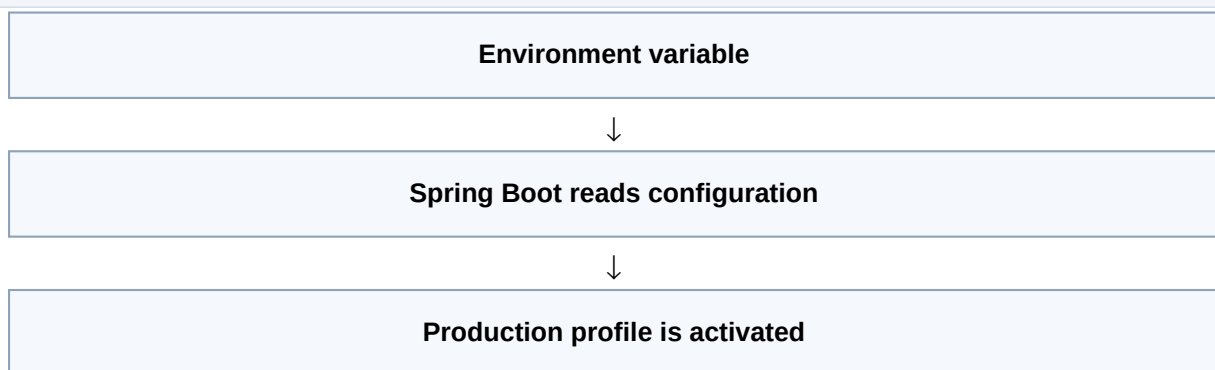
6. Profiles — Loading Flow



7. Environment Variables

- Supply configuration through the environment instead of embedding it in code.
- Useful for deployment-specific values.

```
SPRING_PROFILES_ACTIVE=prod
```



8. @Profile Annotation

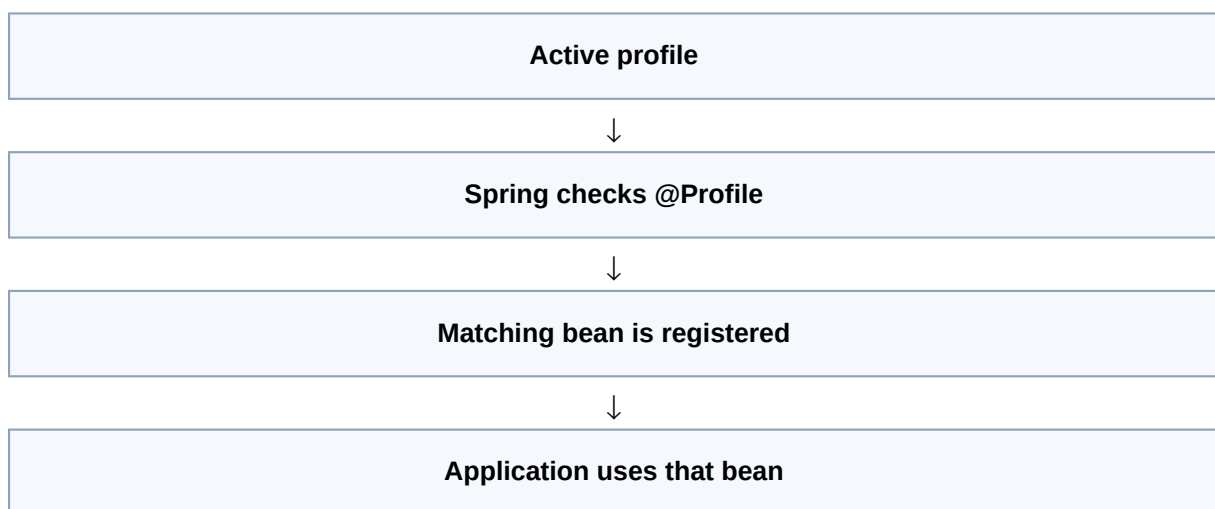
Controls which Spring bean is registered for an active profile.

```
@Component
@Profile("dev")
public class DummyNotificationService {
}
```

```
@Component
@Profile("prod")
public class RealNotificationService {
}
```

- **dev:** Dummy notification service.
- **prod:** Real notification service.
- These are simplified examples of profile-based bean selection.

Bean selection flow:



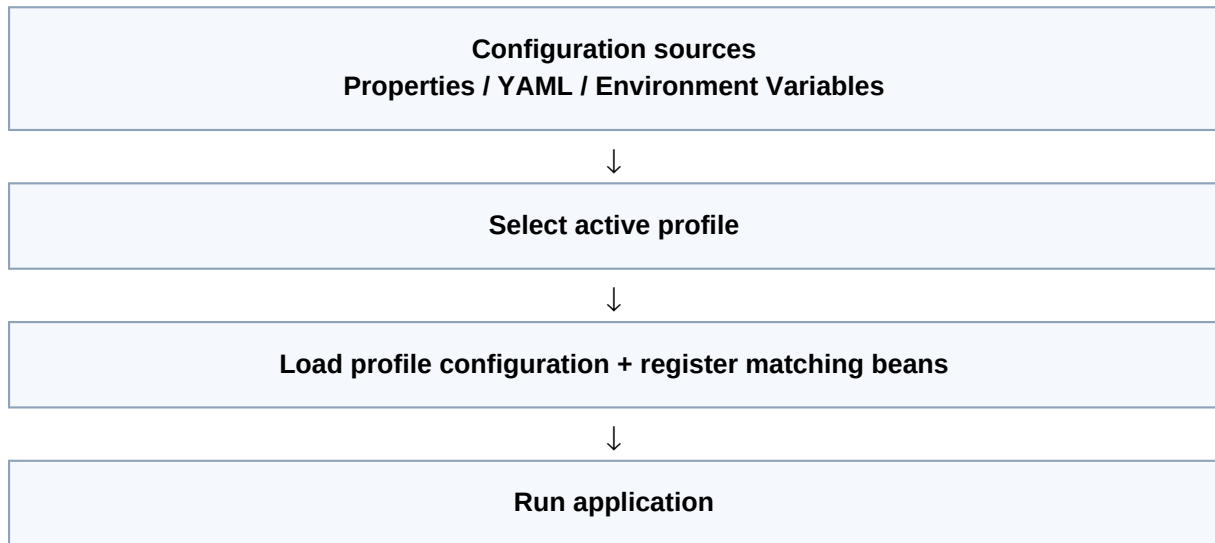
9. Configuration vs @Profile

Feature	Purpose
application-dev.properties	Changes configuration values for dev.
@Profile("dev")	Controls bean registration for dev.
@Value	Injects a configuration value.

10. Common Mistakes

- Hardcoding URLs or credentials in Java code.
- Activating the wrong profile.
- Not providing a matching bean for the active profile.
- Forgetting that required beans must exist for the active profile.
- Committing secrets to source control.

Quick Revision Flow



Remember: Configuration changes values; profiles can change values and which beans are active.